SECURITY

Container Security Visual Guide

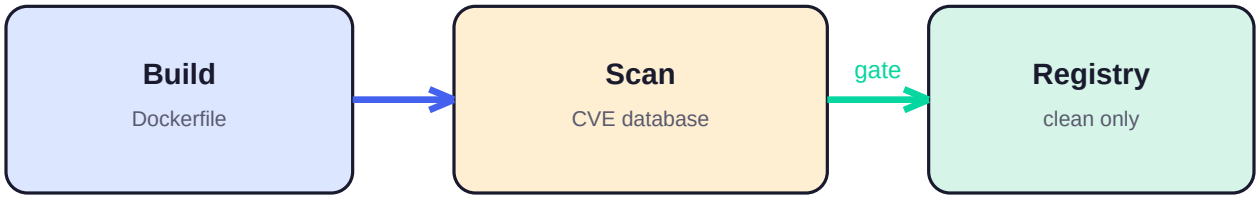
From image to runtime —
lock down every layer of the container stack.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Image Scanning in the Pipeline

Catch vulnerable layers before they deploy.



Fail the build on critical CVEs with fixes available. Rescan on a schedule — new CVEs appear in old images daily.

STEP 2/6

Least-Privilege Users

Containers don't need to be root.

USER root (default)

escape = host root

USER app (uid 1000)

escape = unprivileged

Add 'USER app' to every Dockerfile. A container escape as root owns the host; as an unprivileged user it's a contained nuisance.

STEP 3/6

Read-Only Filesystems

If it can't be written, it can't be backdoored.

- Run containers with read-only root filesystems — malware can't persist.
- Mount writable tmpfs only where the app genuinely needs scratch space.
- Drop /proc and /sys writes; mount secrets as read-only volumes, never env vars in images.
- Verify at deploy: admission policy rejects containers without readOnlyRootFilesystem.

STEP 4/6

seccomp & AppArmor

Shrink the syscalls a container may invoke.

seccomp

Allowlist of syscalls (default profile blocks ~44 dangerous ones).

AppArmor

Path-based mandatory access control per container.

Exploits need syscalls — fewer available syscalls means fewer usable exploit primitives. Enable the defaults everywhere.

STEP 5/6

Never Privileged

The flags that delete your isolation.

- --privileged gives the container nearly all host capabilities — treat as root on the host.
- Drop ALL capabilities, then add back only the minimum (usually none).
- No hostPath mounts, no hostNetwork, no hostPID in multi-tenant clusters.
- Enforce with policy engines (admission controllers) — don't rely on convention.

STEP 6/6

Runtime Protection

Defense doesn't stop at deploy time.

- Runtime scanners watch for unexpected processes, network calls, file writes.
- Image signing + admission: only signed images from your registry may run.
- Network policies: default-deny between namespaces; allow only declared flows.
- Keep the host and runtime patched — container escapes target kernel bugs.

STEP ✓/✓

Key Takeaways

Container Security — Remember

- Scan images in CI and gate on critical CVEs; rescan continuously.
- Run as non-root — 'USER app' in every Dockerfile.
- Read-only filesystems with tmpfs scratch; secrets as volumes, not env.
- Enable seccomp and AppArmor defaults to shrink the attack surface.
- Never --privileged; drop all capabilities; enforce via admission policy.
- Sign images, default-deny network, and monitor runtime behavior.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev